G-Rides

Software Requirements Specification (SRS)

Prepared by GiGOC
Limbe, Cite Sic - Southwest, Cameroon
info@gigoc.org | +237 676 111 147
www.gigoc.org

1. Introduction / Project Overview

G-rides is a digital car rental platform developed under GiGOC. The platform allows users to rent vehicles, list vehicles, manage bookings, track rentals, and coordinate payments through a centralized system.

The platform supports:

• Self-drive rentals
• Chauffeur-assisted rentals
• Vehicle delivery services
• Live rental tracking
• Vehicle listing and management
• Security deposits and verification workflows

2. Purpose of the System

The purpose of G-rides is to provide a secure and organized vehicle rental marketplace in Cameroon.

The system aims to:

• Simplify car rentals
• Increase trust between renters and owners
• Digitize booking and payment workflows
• Reduce fraud and fake listings
• Support dispute management
• Help owners generate income from their vehicles

3. Scope of the System

The system shall support:

• User registration and authentication
• Vehicle listing and management
• Vehicle search and map-based discovery
• Booking and rental management
• Delivery coordination
• Chauffeur support
• Payment and deposit handling
• Verification and reputation systems
• Notifications and communication
• Administrative management

Initial deployment will focus on Cameroon.

4. Stakeholders and User Roles

• Renters – Search and rent vehicles
• Vehicle Owners – List and manage vehicles
• Drivers – Provide chauffeur and delivery services
• Administrators – Manage and monitor the platform
• GiGOC – Own and operate the platform

The platform supports multi-role accounts where one user may act as:
• Renter
• Owner
• Driver

The system also includes:
• Ratings and reviews
• Trust scores
• Rental history
• Dispute records
• Manual owner verification workflows

5. Functional Requirements

Core functional requirements include:
• User registration and authentication
• Vehicle listing creation and editing
• Vehicle availability management
• Vehicle search and filtering
• Booking requests and approvals
• Rental duration tracking
• Delivery requests
• Chauffeur assignment
• Deposit management
• Rental extension requests
• Vehicle condition reporting
• Active rental tracking
• Notifications and alerts
• Review and rating system
• Dispute management
• Administrative moderation

6. Non-Functional Requirements

The system shall provide:
• Secure authentication

- Encrypted communication
- Reliable uptime
- Mobile optimization
- Real-time operations
- Scalable architecture
- API-based communication
- Cross-platform compatibility
- Maintainable backend structure

7. System Architecture and High-Level Design

Frontend:
- React Native

Backend:
- Node.js (NestJS recommended)

Database:
- PostgreSQL

Additional Services:
- Socket.IO for real-time tracking
- Google Maps API for map services
- Cloudinary or AWS S3 for media storage
- Firebase Cloud Messaging for notifications
- Mobile Money and Orange Money integration

8. Database Design / Core Data Entities

Core entities and fields:

1. User
- user_id
- full_name
- phone_number
- email
- password_hash
- profile_photo
- account_status
- created_at
- updated_at

2. User Role

- role_id
- user_id
- role_name
- status

3. Verification
- verification_id
- user_id
- document_type
- document_url
- verification_status
- reviewed_by
- rejection_reason
- submitted_at
- reviewed_at

4. Vehicle
- vehicle_id
- owner_id
- brand
- model
- year
- category
- transmission
- fuel_type
- seat_count
- plate_number
- description
- location_id
- listing_status
- created_at
- updated_at

5. Vehicle Media
- media_id
- vehicle_id
- media_url
- media_type
- is_primary
- uploaded_at

6. Vehicle Documents
- document_id

- vehicle_id
- document_type
- document_url
- verification_status
- reviewed_by
- uploaded_at

7. Vehicle Availability
- availability_id
- vehicle_id
- available_from
- available_to
- is_available
- blocked_reason

8. Location
- location_id
- latitude
- longitude
- address
- city
- region
- last_updated

9. Booking
- booking_id
- renter_id
- vehicle_id
- owner_id
- rental_mode
- booking_status
- start_datetime
- end_datetime
- pickup_option
- delivery_required
- delivery_fee
- total_estimated_cost
- created_at
- updated_at

10. Rental Session
- rental_session_id
- booking_id

• start_time
• expected_end_time
• actual_end_time
• rental_status
• tracking_enabled
• current_location_id
• return_confirmed_by_owner
• return_confirmed_by_renter

11. Payment
• payment_id
• booking_id
• payer_id
• amount
• payment_type
• payment_method
• payment_status
• transaction_reference
• paid_at

12. Driver
• driver_id
• user_id
• license_number
• license_document_url
• driver_status
• availability_status
• rating

13. Driver Assignment
• assignment_id
• booking_id
• driver_id
• assigned_by
• driver_fee
• assignment_status
• assigned_at

14. Review and Rating
• review_id
• booking_id
• reviewer_id
• reviewed_user_id

- rating
- comment
- created_at

15. Dispute
- dispute_id
- booking_id
- raised_by
- dispute_type
- description
- evidence_url
- dispute_status
- resolved_by
- resolution_notes
- created_at
- resolved_at

16. Notification
- notification_id
- user_id
- notification_type
- title
- message
- read_status
- created_at

17. Audit Log
- log_id
- user_id
- action
- entity_type
- entity_id
- ip_address
- created_at

18. Vehicle Condition Report
- condition_report_id
- booking_id
- vehicle_id
- submitted_by
- report_type
- notes

• media_urls
• created_at

9. Booking and Rental Workflow

Rental workflow:
1. Vehicle Search
2. Booking Request
3. Booking Review
4. Payment and Verification
5. Booking Confirmation
6. Vehicle Handover
7. Active Rental
8. Rental Return
9. Deposit Handling
10. Rental Completion

The system supports:
• Digital handover confirmation
• Delivery workflows
• Chauffeur workflows
• Rental extension workflows
• Cancellation handling
• Dispute handling

10. Payment, Deposit, and Refund Policies

The system shall support:
• Booking fees
• Rental payments
• Delivery fees
• Chauffeur fees
• Security deposits
• Refunds

Important rules:
• Deposits remain held until rental completion
• Owners cannot withdraw held deposits
• Withdrawals require approval conditions
• The platform supports controlled payout handling
• Rental payments and deposits may remain held during disputes

The platform includes:

• Internal owner wallet balances
• Available balances
• Held balances
• Withdrawal requests

11. Security, Verification, and Trust Management

Security features include:
• Identity verification
• Driver verification
• Manual owner verification
• Vehicle document validation
• Listing approval workflows
• Ratings and reviews
• Trust scores
• Digital rental agreements
• Vehicle condition reports
• Active rental tracking
• Secure payment handling
• Controlled withdrawals
• Fraud monitoring
• Role-based access control
• Admin activity logging

12. Future Expansion

The platform architecture is designed to support future expansion into:
• Bike rentals
• Van rentals
• Truck rentals
• Logistics services
• Delivery systems
• Fleet management
• Driver marketplaces
• Transport operator services